

Bài 4: Lập trình trên máy tính



Mục tiêu

- Hiểu khái niệm về chương trình (Program) và cách phát triển chương trình
- Hiểu sự khác biệt giữa ngôn ngữ cấp thấp (low-level language) và ngôn ngữ cấp cao (high-level language)
- Giới thiệu về ngôn ngữ cấp thấp sử dụng ngôn ngữ lập trình Assembly làm ví dụ
- Tìm hiểu về cấu trúc của chương trình, thuật toán (algorithms), mã giả (pseudocode) và lưu đồ (flowchart)

Chương trình (Program) là gì?

- Là một tập hợp các câu lệnh dùng để giải quyết một vấn đề.
- Các câu lệnh này được nhập theo thứ tự logic (còn được gọi là thuật toán (Algorithm))
- Muốn máy tính chạy được chương trình thì những câu lệnh này cần phải chuyển đổi thành ngôn ngữ mà máy tính hiểu được
- Quá trình chuyển đổi sử dụng trình thông dịch hoặc trình biên dịch
 - Trình thông dịch dịch các câu lệnh một cách tuần tự
 - Trình biên dịch đọc tất cả các câu lệnh và tạo ra một chương trình hoàn chỉnh.

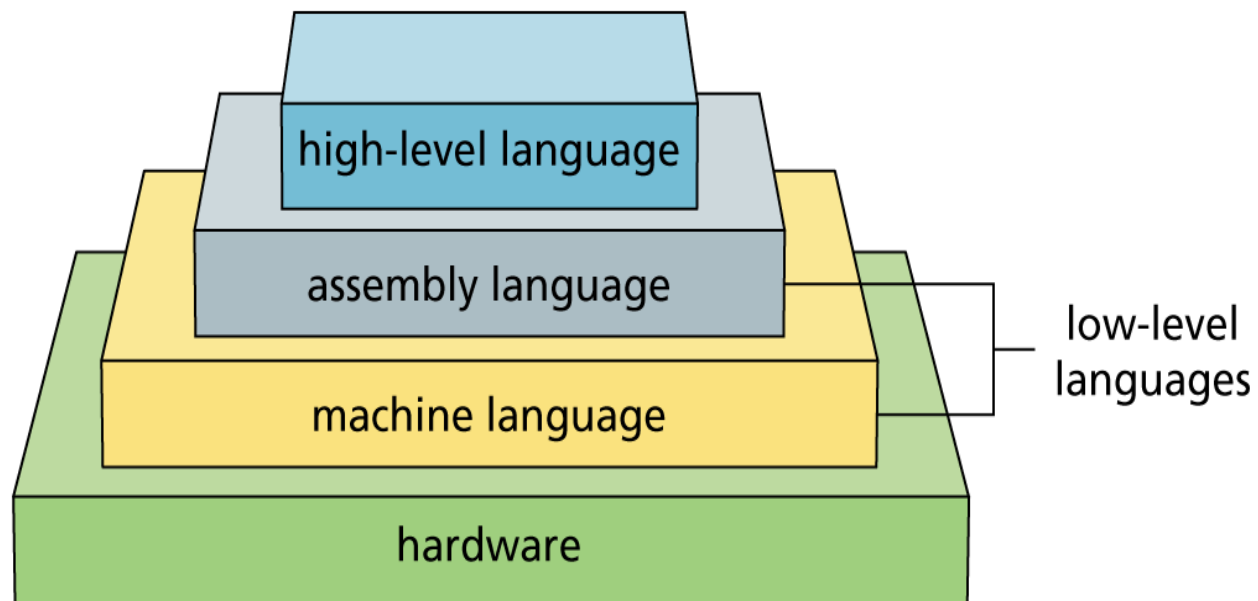
Giao tiếp với máy tính

- Bước đầu tiên trong lập trình là xác định ngôn ngữ mà bạn muốn sử dụng để giao tiếp với máy tính.
- Một vài ngôn ngữ mà bạn có thể lựa chọn:
 - Ngôn ngữ Assembly để điều khiển phần cứng
 - Ngôn ngữ Java và JavaScript cho các ứng dụng Internet
 - Ngôn ngữ Lisp để làm việc với trí tuệ nhân tạo
 - Ngôn ngữ Visual Basic cho một môi trường lập trình giao diện đồ họa đơn giản nhưng mạnh mẽ
 - Các ngôn ngữ khác bao gồm: C, C++, Smalltalk, Delphi, và ADA, FORTRAN, và COBOL

Các loại Ngôn ngữ lập trình (Programming Language)

- Ngôn ngữ cấp thấp (low-level language)
 - Hướng tới máy tính - ít hiểu được hoặc giống ngôn ngữ con người
 - Ngôn ngữ máy (Machine language) là ngôn ngữ cấp thấp nhất
 - Ngôn ngữ hợp ngữ (Assembly) nằm giữa ngôn ngữ cấp thấp nhất và ngôn ngữ cấp cao hơn
 - Trình biên dịch Assembler được sử dụng để chuyển đổi mã lệnh viết bằng ngôn ngữ hợp ngữ (assembly code) thành mã máy (machine code).
- Cấp cao (high-level language)
 - Ngôn ngữ dễ hiểu cho con người

Các loại Ngôn ngữ lập trình (Programming Language) (tt)



Ngôn ngữ cấp thấp (Low-level language)

- Ngôn ngữ máy chỉ sử dụng số nhị phân (0, 1).
- Ngôn ngữ hợp ngữ (Ngôn ngữ Assembly) sử dụng các câu lệnh tựa tiếng Anh.
 - Mỗi câu lệnh tương ứng với một sự hướng dẫn cho máy tính.
 - Chương trình viết bằng ngôn ngữ cấp thấp chạy nhanh hơn so với chương trình viết bằng ngôn ngữ cấp cao.
 - Liên quan chặt chẽ đến loại CPU cụ thể.
 - Khó đọc và khó hiểu hơn so với ngôn ngữ cấp cao.

Các câu lệnh của ngôn ngữ hợp ngữ (Assembly)

- Mỗi câu lệnh trong Assembly bao gồm các thành phần sau:
 - Chỉ thị (Instruction): Đại diện cho hoạt động cần thực hiện, ví dụ như mov (di chuyển), add (cộng), sub (trừ), jmp (nhảy),...
 - Đích (Destination): Là nơi mà kết quả của hoạt động sẽ được lưu trữ, thường là một thanh ghi hoặc một vị trí bộ nhớ.
 - Nguồn (Source): Là giá trị hoặc địa chỉ mà câu lệnh sẽ sử dụng để thực hiện hoạt động, có thể là một thanh ghi, một hằng số hoặc một vị trí bộ nhớ.

Các câu lệnh của ngôn ngữ hợp ngữ (Assembly) (tt)

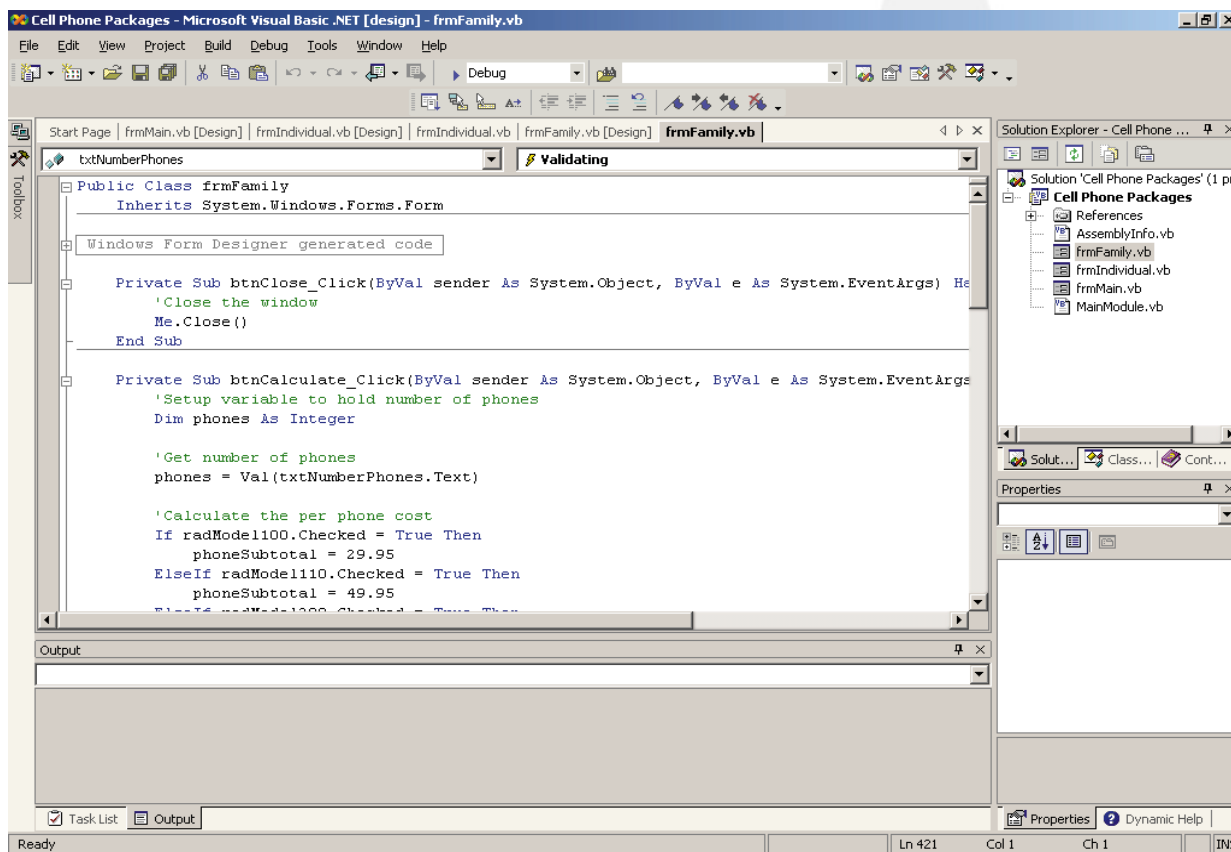
- Ví dụ:

```
mov cx, 3    //sao chép (copy) giá trị 3 vào thanh ghi cx → cx=3
mov dx, 8    //sao chép (copy) giá trị 8 vào thanh ghi dx → dx=8
add dx, cx   //lấy giá trị 3 trong thanh ghi cx cộng với giá trị 8 trong
             //thanh ghi dx, kết quả 11 ghi vào thanh ghi dx → dx=11
inc dx       //tăng giá trị trong thanh ghi dx lên 1 → dx=12
sub dx, cx   //lấy giá trị trong thanh ghi dx trừ giá trị trong thanh
             //ghi cx, kết quả trừ lưu vào thanh ghi dx → dx=9
cmp dx, cx   //so sánh giá trị của dx với cx → dx có giá trị khác cx nên
             //biến cờ ZF=0
jnz stop     // lệnh nhảy (go to) tới nơi có nhãn "stop" trong chương
             //trình khi biến cờ ZF = 1
```

Ngôn ngữ cấp cao (high-level language)

- Dễ viết, đọc và bảo trì hơn so với ngôn ngữ cấp thấp.
- Hoàn thành nhiều công việc hơn trong một câu lệnh duy nhất.
- Thường có tốc độ chạy chậm hơn so với ngôn ngữ cấp thấp.
- Phải được biên dịch hoặc thông dịch.
- Có nhiều IDE (integrated development environment) kết hợp để cung cấp trình soạn thảo, trình biên dịch, thiết kế đồ họa và nhiều tính năng khác.

Ngôn ngữ cấp cao (high-level language) (tt)



Một IDE giúp phát triển phần mềm dễ dàng hơn

Cấu trúc của một chương trình (Structure of a Program)

- Là cách tổ chức và sắp xếp các phần của một chương trình máy tính để nó có thể thực thi các tác vụ cụ thể. Cấu trúc chương trình dựa trên Giải thuật (Algorithm).
- Có bốn loại cấu trúc trong ngôn ngữ cấp cao:
 - Cấu trúc gọi hàm (Invocation)
 - Cấu trúc tuần tự (Top-down)
 - Cấu trúc lựa chọn (Selection)
 - Cấu trúc lặp (Repetition)

Cấu trúc gọi hàm (Invocation structure)

- Viết chương trình theo dạng tách hàm.
- Mỗi hàm thực hiện 1 tác vụ cụ thể.
- Có thể tái sử dụng mã (code) của hàm nhiều lần trong cùng 1 chương trình, hoặc ở 1 chương trình khác.

Cấu trúc gọi hàm (Invocation structure) (tt)

```
#include <stdio.h>

// Khai báo hàm printMessage
void printMessage();

int main() {
    // Gọi hàm printMessage ba lần
    printMessage();
    printMessage();
    printMessage();

    return 0;
}

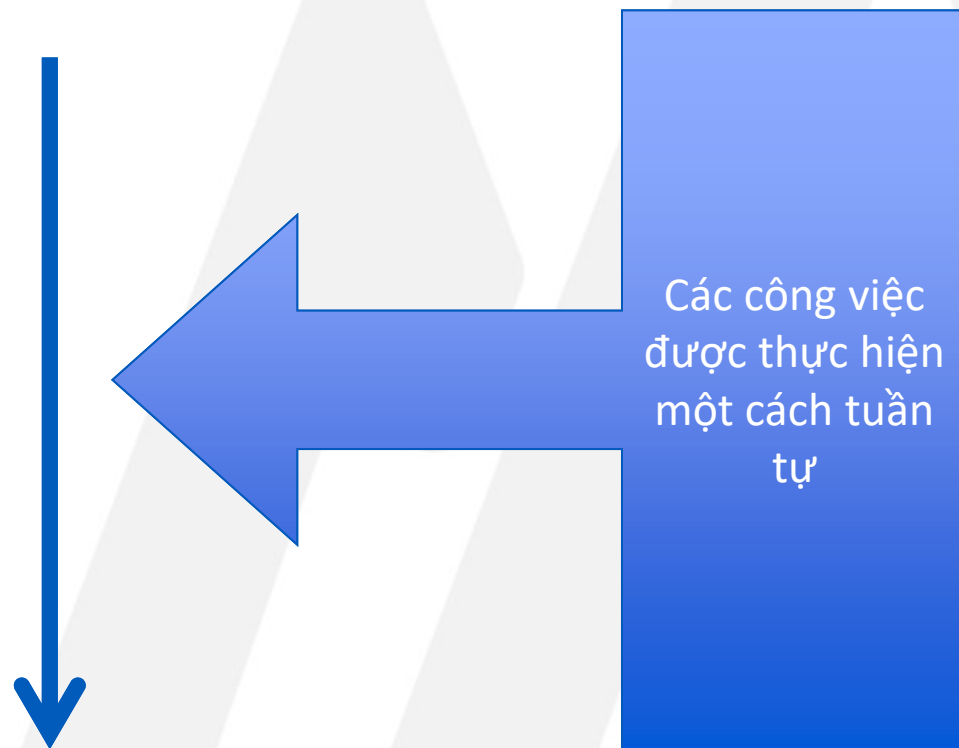
// Định nghĩa hàm printMessage
void printMessage() {
    printf("Hello, world!\n");
}
```


Cấu trúc tuần tự (Top-down / Sequence structure)

- Các câu lệnh trong chương trình được thực thi theo thứ tự, từ dòng trên cùng đến dòng dưới cùng, mỗi lần thực hiện một dòng.
- Đây là hình thức phổ biến nhất của cấu trúc điều khiển trong lập trình, được tìm thấy trong mọi ngôn ngữ lập trình.
- Cấu trúc này được thực hiện bằng cách gõ vào các câu lệnh không gọi các phần mã nguồn khác.

Cấu trúc tuần tự (Top-down / Sequence structure) (tt)

- Công việc 1
- Công việc 2
- Công việc 3
- Công việc 4
- ...
- Công việc N



Cấu trúc tuần tự (Top-down / Sequence structure) (tt)

- Bước 1: Chuẩn bị nguyên liệu
- Bước 2: Ngâm nếp
- Bước 3: Gói bánh
- Bước 4: Nấu bánh
- Bước 5: Vớt bánh



**Các bước làm
bánh tét**

Cấu trúc tuần tự (Top-down / Sequence structure) (tt)

```
#include <stdio.h>

int main() {
    // Khai báo và khởi tạo biến
    int number1 = 5;
    int number2 = 10;

    // Thực hiện phép tính
    int sum = number1 + number2;

    // In kết quả ra màn hình
    printf("Tổng hai số là: %d\n", sum);

    // Thực hiện câu lệnh tiếp theo
    printf("Chương trình kết thúc.\n");

    return 0;
}
```

Cấu trúc lựa chọn (Selection structure)

- Cho phép lựa chọn giữa hai hoặc nhiều khối mã lệnh để thực thi, tùy thuộc vào giá trị của một biểu thức hoặc điều kiện.

Cấu trúc lựa chọn (Selection structure) (tt)

```
#include <stdio.h>

int main() {
    int điểm;

    printf("Nhập điểm trung bình: ");
    scanf("%d", &điểm);

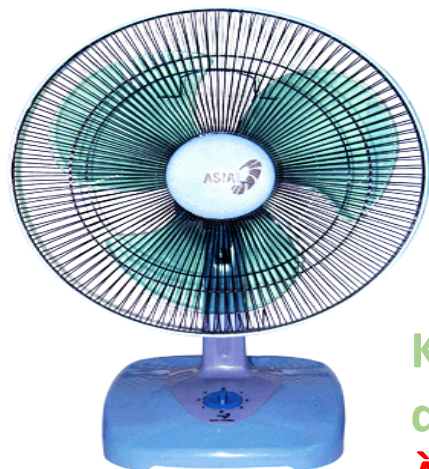
    if (điểm >= 5) {
        printf("Chúc mừng. Bạn đã đậu");
    }
    else {
        printf("Bạn không đủ điểm. Cố gắng lần sau");
    }

    return 0;
}
```


Cấu trúc Lặp (Repetition structure)

- Cho phép chương trình lặp đi lặp lại một khối mã lệnh nhiều lần, dựa trên một điều kiện được xác định.
- Tiết kiệm công sức so với việc viết lại mã lệnh nhiều lần.

Cấu trúc Lặp (Repetition structure)



Khi nào thì cái
quạt hết quay?
**À khi chúng ta
ngắt điện**



Khi nào cái đồng
hồ ngừng chạy?
À khi hết pin

Cấu trúc Lặp (Repetition structure) (tt)

```
#include <stdio.h>

int main() {
    int i;

    // Sử dụng vòng lặp for để in ra 1000 dòng "Hello"
    for (i = 1; i <= 1000; i++) {
        printf("Hello\n");
    }

    return 0;
}
```

GIẢI THUẬT (Algorithm)

- **Giải thuật** còn gọi là **Thuật toán**, là các bước thực thi **rõ ràng**, **hữu hạn**, **có trình tự** thực hiện bài toán từ trạng thái bắt đầu đến khi đạt được kết quả mong muốn cuối cùng.
- Lưu ý: cùng một giải thuật, nếu thực hiện n lần **khác nhau**, sẽ cho cùng một kết quả **giống nhau**

GIẢI THUẬT (tt)

Ví dụ: giải thuật giải phương trình bậc nhất $ax+b=0$, lần lượt thực hiện như sau:

- Nếu a **bằng** 0
 - nếu b **bằng** 0 thì phương trình vô số nghiệm
 - nếu b **khác** 0 thì phương trình vô nghiệm
- Nếu a **khác** 0
 - phương trình có nghiệm duy nhất $x = (-b/a)$

* Các bước làm ở trên để giải PT bậc nhất gọi là ngôn ngữ giải thuật hay còn gọi là **mã giả (Pseudocode)**

MÔ TẢ GIẢI THUẬT

Có hai cách mô tả giải thuật:

- Mô tả giải thuật bằng ngôn ngữ tự nhiên, gần giống với ngôn ngữ lập trình, chúng ta gọi là **mã giả (pseudocode)**.
- Mô tả giải thuật bằng hình vẽ, chúng ta gọi là **lưu đồ (Flowchart)**.

MÃ GIẢ (pseudocode)

- Là một mô tả giải thuật lập trình máy tính ngắn gọn bằng các quy ước có cấu trúc của một số ngôn ngữ lập trình, nhưng bỏ đi những chi tiết không cần thiết để hiểu rõ giải thuật hơn.

CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ

TUẦN TỰ:

- **Input:** READ, INPUT, OBTAIN, GET
- **Output:** DISPLAY, PRINT, SHOW
- **Compute:** CALCULATE, COMPUTE, DETERMINE
- **Initialize:** SET, INIT
- **Add one:** INCREMENT

CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ (tt)

Rẽ nhánh IF-ELSE

IF condition THEN

sequence_1

ELSE

sequence_2

ENDIF

```
BEGIN
```

```
    INPUT Diem
```

```
    IF Diem >= 5 THEN
```

```
        PRINT "Dau"
```

```
    ELSE
```

```
        PRINT "Rot"
```

```
    ENDIF
```

```
END
```

CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ (tt)

Rẽ nhánh CASE

CASE expression

condition_1: sequence_1

condition_2: sequence_2

condition_3: sequence_3

....

OTHERS:

sequence_n

ENDCASE

BEGIN

INPUT So

CASE So

1: PRINT "Chu nhật"

2: PRINT "Thứ hai"

3: PRINT "Thứ ba"

4: PRINT "Thứ tư"

5: PRINT "Thứ năm"

6: PRINT "Thứ sáu"

7: PRINT "Thứ bảy"

OTHERS:

PRINT "Số không hợp lệ"

ENDCASE

END

CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ (tt)

Vòng lặp WHILE

WHILE condition

sequence

ENDWHILE

```
BEGIN
```

```
    SET Counter =1
```

```
    WHILE Counter <=10
```

```
        Print "Hello"
```

```
        INCREMENT Counter
```

```
    ENDWHILE
```

```
END
```

CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ (tt)

Vòng lặp DO WHILE

DO

sequence

WHILE condition

```
BEGIN
    DO
        DISPLAY "Hãy nhập điểm số: "
        INPUT diem
    WHILE diem < 0 || diem > 10

    IF diem < 5 THEN
        DISPLAY "Rot"
    ELSE
        DISPLAY "Dau"
    ENDIF
END
```


CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ (tt)

Vòng lặp REPEAT-UNTIL

REPEAT

sequence

UNTIL condition

```
BEGIN
    REPEAT
        DISPLAY "Hãy nhập điểm số: "
        INPUT diem
    UNTIL diem >=0 && diem <=10

    IF diem < 5 THEN
        DISPLAY "Rot"
    ELSE
        DISPLAY "Dau"
    ENDIF
END
```

CÁC TỪ KHÓA SỬ DỤNG TRONG MÃ GIẢ (tt)

Vòng lặp FOR

FOR i= init TO end

sequence

ENDFOR

```
BEGIN
```

```
    FOR i=1 TO 10
```

```
        Print "Hello"
```

```
    ENDFOR
```

```
END
```

VÍ DỤ MÃ GIẢ

Ví dụ 1:

Dùng mã giả để mô tả giải thuật nhập 2 số nguyên a , b . Xuất ra tổng, hiệu, tích thương.

BEGIN

INPUT a

INPUT b

tong = $a+b$

hieu = $a-b$

tich = $a*b$

IF $b \neq 0$ THEN

thuong = a/b

ENDIF

DISPLAY tong

DISPLAY hieu

DISPLAY tich

DISPLAY thuong

END

VÍ DỤ MÃ GIẢ

Ví dụ 2:

Dùng mã giả để mô tả giải thuật nhập vào số nguyên dương N. Tính tổng $S=1+2+3+\dots+N$

BEGIN

DO

INPUT N

WHILE (N<=0) //N phải >0

S = 0

FOR i=1 TO N

S = S+i

ENDFOR

DISPLAY S

END

LƯU ĐỒ GIẢI THUẬT

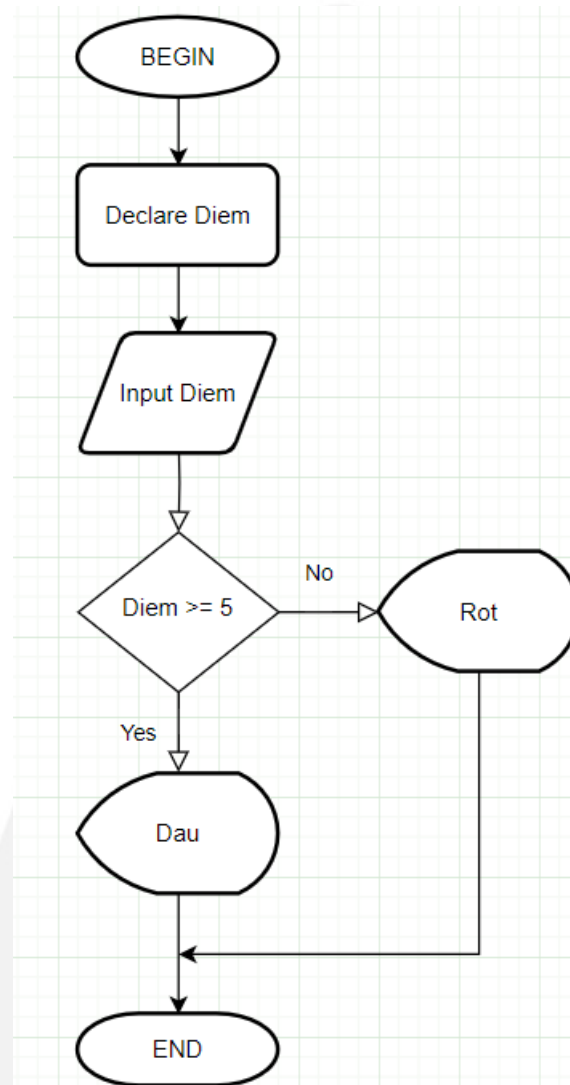
Lưu đồ giải thuật dùng để diễn tả giải thuật bằng các khối hình học. (Xem thêm giải thích các ký hiệu [tại đây](#))

Flowchart Symbol	Name	Flowchart Symbol	Name
	Process symbol		Connector symbol
	Start/End symbol		Off-Page Connector/Link symbol
	Document symbol		Input/Output symbol
	Decision symbol		Comment/Note symbol
	Display		Flow line

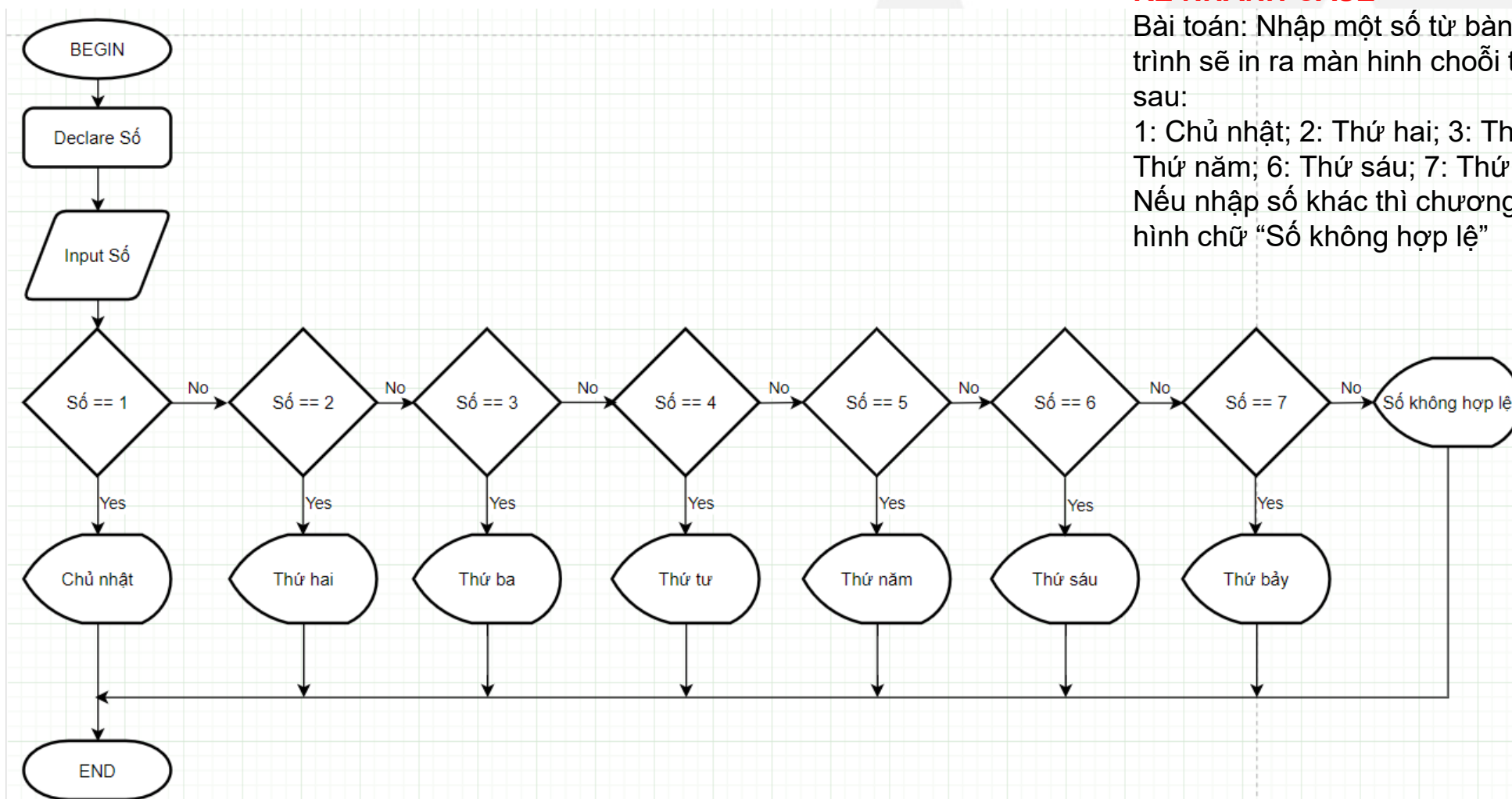
LƯU ĐỒ GIẢI THUẬT

RỄ NHÁNH IF...ELSE

Bài toán: Nhập điểm số từ bàn phím. Nếu điểm lớn hơn hoặc bằng 5 thì in ra màn hình choãi "Dau", ngược lại in chữ "Rot"



LƯU ĐỒ GIẢI THUẬT



RỄ NHÁNH CASE

Bài toán: Nhập một số từ bàn phím. Chương trình sẽ in ra màn hình choỗi tương ứng như sau:

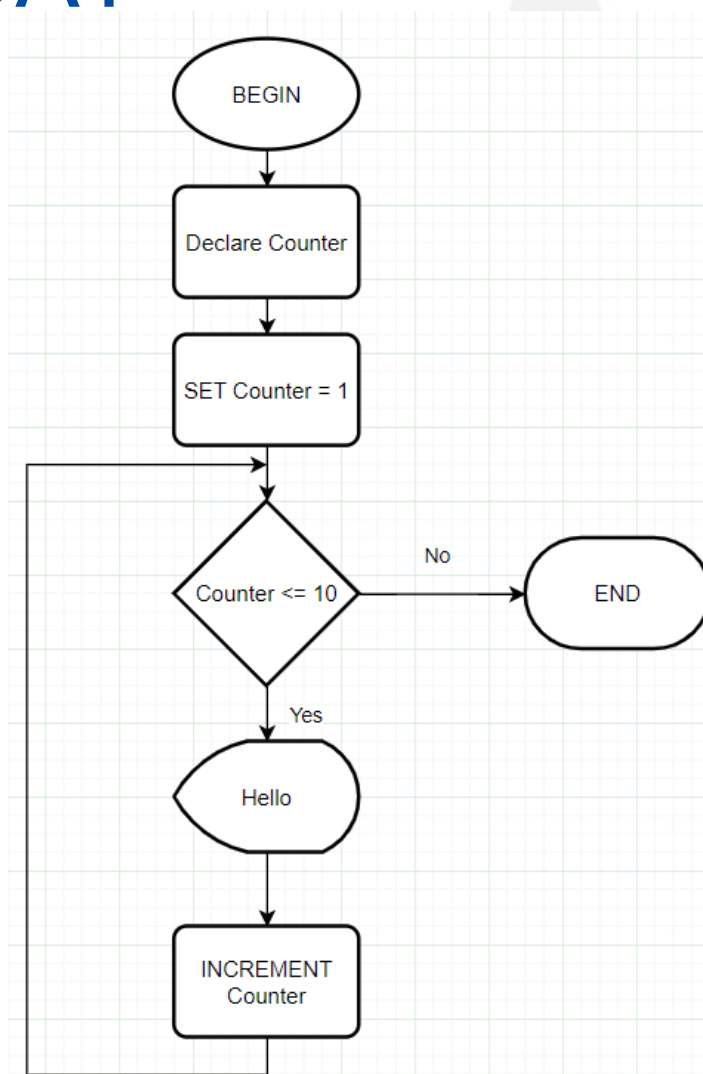
1: Chủ nhật; 2: Thứ hai; 3: Thứ ba; 4: Thứ tư; 5: Thứ năm; 6: Thứ sáu; 7: Thứ bảy

Nếu nhập số khác thì chương trình sẽ in ra màn hình chữ "Số không hợp lệ"

LƯU ĐỒ GIẢI THUẬT

VÒNG LẶP WHILE

Bài toán: In ra màn hình 10 dòng chữ Hello. Sử dụng vòng lặp WHILE



LƯU ĐỒ GIẢI THUẬT

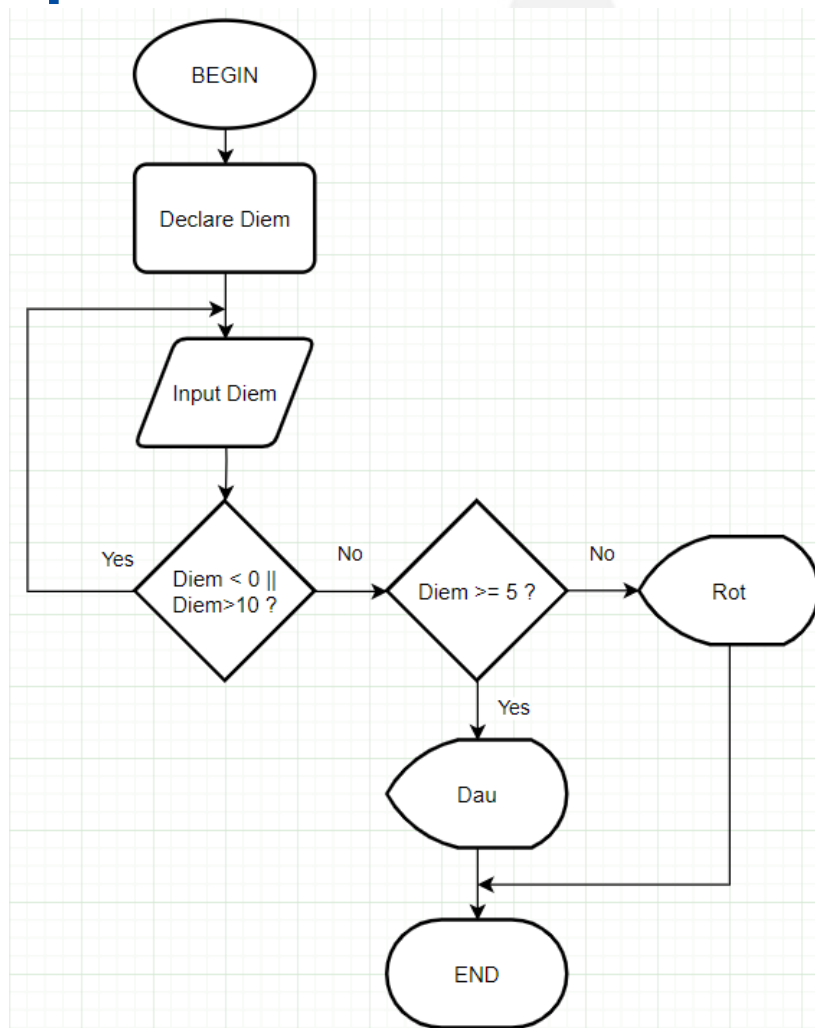
VÒNG LẶP DO...WHILE

Bài toán:

+ Nhập Điểm từ bàn phím. Nếu Điểm nằm ngoài khoảng $[0, 10]$ thì yêu cầu nhập lại.

+ Căn cứ vào Điểm vừa nhập, nếu $\text{Điểm} \geq 5$ thì in ra màn hình chữ “Đậu”, ngược lại in chữ “Rớt”

Gợi ý: Sử dụng vòng lặp DO...WHILE



LƯU ĐỒ GIẢI THUẬT

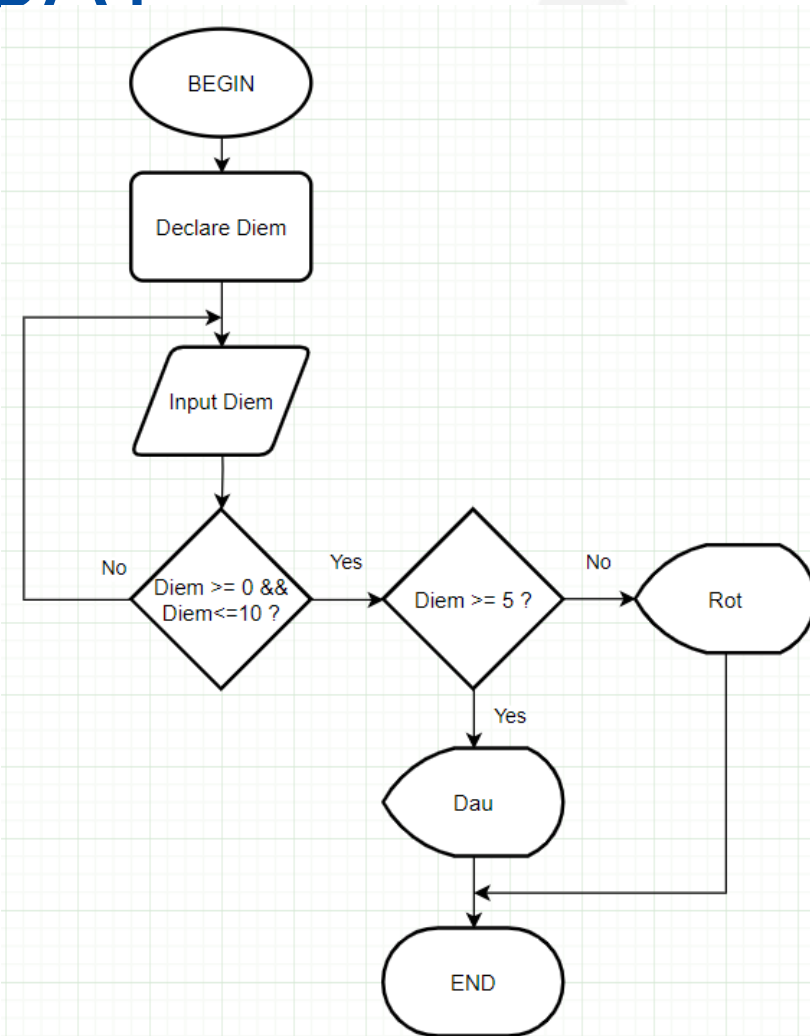
VÒNG LẶP REPEAT...UNTIL

Bài toán:

+ Nhập Điểm từ bàn phím. Nếu Điểm nằm ngoài khoảng $[0, 10]$ thì yêu cầu nhập lại.

+ Căn cứ vào Điểm vừa nhập, nếu $\text{Điểm} \geq 5$ thì in ra màn hình chữ “Đậu”, ngược lại in chữ “Rớt”

Gợi ý: Sử dụng vòng lặp REPEAT ...UNTIL



LƯU ĐỒ GIẢI THUẬT

VÒNG LẶP FOR

Bài toán: In ra màn hình 10 dòng chữ Hello. Sử dụng vòng lặp FOR

